

Services, Data Sharing & Forms

shared logic, dependency injection, and handling user input

DR

Dinesh Rawat

Instructor · Shorter Loop × Snapied

Two big topics. First **services**: shared logic, dependency injection, and singletons that let components share data. Then **forms**: the two ways Angular handles user input, template-driven and reactive, with small examples of each.

1 What is a service?

A **service** is a plain class that holds logic or data you want to reuse across components. Fetching from an API, storing the logged-in user, a calculation many places need: all of this belongs in a service, not repeated inside each component.

In one line: components are for the view, services are for the shared work. Keep components light, put reusable logic in services.

2 Making a service

A service is a class marked with `@Injectable`. The CLI makes one: `ng generate service data`.

DATA.SERVICE.TS

```
import { Injectable } from "@angular/core";

@Injectable({ providedIn: "root" })
export class DataService {
  private message: string = "Hello from the service";

  getMessage(): string { return this.message; }
  setMessage(newMessage: string): void { this.message = newMessage; }
}
```

`@Injectable` marks the class as injectable. `providedIn: "root"` makes it available across the whole app.

3 Dependency injection: how a component gets a service

A component does not create the service itself. It **asks** for it in the constructor, and Angular **hands it over**. This is **dependency injection** (DI).

HELLO.COMPONENT.TS

```
import { Component, OnInit } from "@angular/core";
import { DataService } from "../data.service";

@Component({ selector: "app-hello", template: `<p>{{ text }}</p>` })
export class HelloComponent implements OnInit {
  text: string = "";

  constructor(private dataService: DataService) {}

  ngOnInit() {
    this.text = this.dataService.getMessage();
  }
}
```

The constructor line `private dataService: DataService` says "I need a DataService." Angular creates it (or reuses the existing one) and passes it in. You never write `new DataService()` yourself.

4 Singleton and data sharing

When a service is `providedIn: "root"`, Angular makes **one instance** for the whole app and gives that same one to everybody. One shared instance is called a **singleton**.

Why this matters: because everyone shares the same instance, if one component changes a value in the service, every other component sees it. That is how two unrelated components share data:

```
// Component A writes:
this.dataService.setMessage("Updated by A");

// Component B reads:
this.text = this.dataService.getMessage(); // "Updated by A"
```

When to use what: `@Input` and `@Output` for a directly connected parent and child. A **service** for components anywhere in the app that need to share data.

5 Forms: two ways to handle input

Almost every app has forms: login, signup, search. Angular gives you **two ways** to build them:

	Template-driven	Reactive
Where the logic lives	mostly in the HTML template	mostly in the TypeScript class
Module needed	<code>FormsModule</code>	<code>ReactiveFormsModule</code>
Best for	simple, small forms	bigger forms, complex validation
Feel	quick and easy	more control, more code

Both are valid. Learn template-driven first because it is simpler, then reactive because it is what most professional projects use.

6 Template-driven forms

Here the form is built in the HTML, using `ngModel` for two-way binding. The class stays almost empty. You need `FormsModule`.

THE TEMPLATE

```
<form (ngSubmit)="onSubmit()">
  <input name="username" [(ngModel)]="username" placeholder="Name" />
  <input name="email" [(ngModel)]="email" placeholder="Email" />
  <button type="submit">Submit</button>
</form>
```

THE CLASS

```
username: string = "";
email: string = "";

onSubmit() {
  console.log(this.username, this.email);
}
```

Each input is tied to a class property with `[(ngModel)]`. When the form is submitted, `onSubmit()` runs and the values are already in the class. Simple and fast for small forms.

7 Reactive forms

Here the form is built in the **class**, using `FormGroup` and `FormControl`. The template just connects to it. You get more control, which helps for bigger forms and validation. You need `ReactiveFormsModule`.

THE CLASS

```
import { Component } from "@angular/core";
import { FormGroup, FormControl, Validators } from "@angular/forms";

@Component({ /* ... */ })
export class SignupComponent {
  signupForm = new FormGroup({
    username: new FormControl("", Validators.required),
    email: new FormControl("", [Validators.required, Validators.email]),
  });

  onSubmit() {
    console.log(this.signupForm.value);
  }
}
```

THE TEMPLATE

```
<form [formGroup]="signupForm" (ngSubmit)="onSubmit()">
  <input formControlName="username" placeholder="Name" />
  <input formControlName="email" placeholder="Email" />
  <button type="submit" [disabled]="signupForm.invalid">Submit</button>
</form>
```

The form is defined in the class as a `FormGroup` of `FormControl`s. The template links to it with `[formGroup]` and `formControlName`. Notice validation is built in: `Validators.required` and `Validators.email`, and the button disables itself while the form is invalid.

8 Validation, in short

Reactive forms make validation easy. Common validators:

- `Validators.required` – the field cannot be empty.
- `Validators.email` – must be a valid email.
- `Validators.minLength(6)` – at least 6 characters.
- `Validators.maxLength(20)` – at most 20 characters.

You can check the form's state anytime: `signupForm.valid`, `signupForm.invalid`, or a single field with `signupForm.get('email')?.invalid`.

9 Remember this

Service: a class for shared logic and data. `@Injectable` with `providedIn: "root"` .

Dependency injection: ask for a service in the constructor, Angular provides it. Never write `new` .

Singleton: one shared instance for the whole app. This is how components share data through a service.

Template-driven forms: built in the HTML with `ngModel` , needs `FormsModule` . Best for small forms.

Reactive forms: built in the class with `FormGroup` and `FormControl` , needs `ReactiveFormsModule` . Best for bigger forms and validation.

10 Homework

Make a service with `ng g s data` , inject it into a component, and show its message.

Prove the singleton: inject it into two components, set a value in one, read it in the other.

Build a template-driven form with two fields and log the values on submit.

Build a reactive form with the same fields, add `Validators.required` , and disable the submit button while the form is invalid.